Course Name: Deep Learning

Course Code: IS3332-1

Title of the Assignment: CNN + RNN + GAN (Applied Deep Models)

Assignment Number: 2

Name: K H Sushanth

USN: NNM23IS083

Section & Semester: 6th 'B'

Department: Information Science and Engineering

Institution Name: NMAM Institute of Technology, Nitte

Academic Year: 2025–26

Submitted to: Dr. Jason Elroy Martis, Associate Professor

Date of Submission:22-3-2026

2. ABSTRACT

The focus of this assignment is to implement and analyze deep learning models for image classification, sequence learning, and generative models using the PyTorch library. Three major models were developed, which included a Convolutional Neural Network (CNN) for image classification using the CIFAR-10 dataset, a Recurrent Neural Network and its variations for sequence learning using the IMDB dataset, and a Generative Adversarial Network for generating images using the Fashion-MNIST dataset.

The CNN model was found to have a decent accuracy rate, ranging from 75% to 81%. The performance was improved using a ResNet18 model with transfer learning, provided the hyperparameters were tuned accordingly. The GRU and LSTM models had a superior performance over the standard RNN model. The GAN model was found to generate images, with quality improving over time.

3. GITHUB LINK

https://github.com/KHSushanth/deeplearning-_task2

4. INTRODUCTION

Deep learning is considered to be one of the most dominant paradigms of the modern concept of artificial intelligence, which enables computers to learn complex patterns from the data fed into the system. Unlike traditional machine learning models, deep learning models can learn features directly from the data without any prior processing of the data.

The tremendous growth of computational power, availability of large-scale data, and the emergence of efficient frameworks such as PyTorch have propelled the use of deep learning models in different domains such as healthcare, autonomous systems, recommendation systems, etc.

This assignment is centered around the implementation and analysis of the three major categories of deep learning models. These include:

1. Convolutional Neural Networks (CNN)

Convolutional neural networks have been designed for the processing of grid-like data, especially images. CNNs have the capability of automatically extracting spatial features from images. In this project, a custom CNN model is implemented for image classification using the CIFAR-10 dataset. In addition, transfer learning is used with a ResNet18 model for image classification.

2. Recurrent Neural Networks (RNN, LSTM, GRU)

Recurrent neural networks have been used for sequential data processing. However, the traditional RNN model has limitations, especially the "Vanishing Gradient" issue. To mitigate this, advanced RNN models like the Long Short-Term Memory (LSTM) model and the Gated Recurrent Unit (GRU) model have been proposed. In this project, the RNN models have been used for sentiment analysis using the IMDB dataset.

3. Generative Adversarial Networks (GAN)

Generative Adversarial Networks have been proposed as a class of deep neural network architectures used for the generation of new samples of data that have similarities with the original dataset. In this project, a GAN model is proposed for the generation of images of clothing items using the Fashion MNIST dataset.

5. PROBLEM STATEMENT

The main idea behind this assignment is to develop, execute, and test different deep learning models for solving real-life problems in the following three main areas: image classification, sequence learning, and generation. This will help us understand the efficiency of different architectures of neural networks on different datasets.

.

The first task is to implement a Convolutional Neural Network (CNN) to classify the CIFAR-10 image dataset. The aim is to classify the image correctly into any of the ten different object categories. The experiment is conducted using a custom CNN model as well as the transfer learning method using the ResNet18 model. The models are evaluated on the basis of their accuracy as well as their behavior during training.

The second task entails sequence learning for text classification with the help of the IMDB Dataset. Three models of recurrent neural networks, namely RNN, LSTM, and GRU, will be implemented to perform a sentiment analysis task on movie reviews and classify them as positive or negative.

The third task entails designing a Generative Adversarial Network model to generate synthetic images with the help of the Fashion-MNIST Dataset.

The expected outcomes include the successful training of all models and their performance evaluation with the help of suitable parameters. Assumptions and constraints include the assumption of enough computational power and a general assumption of data preprocessing. Constraints include limited time and computational power available for model training and the challenges of training deep neural networks, which include stability issues with GAN models and overfitting with classification models..

6. DATASET DESCRIPTION

This project uses three different datasets corresponding to the three implemented deep learning tasks: image classification, sequence learning, and generative modeling.

6.1 CIFAR-10 Dataset

The CIFAR-10 dataset is a widely used benchmark dataset for image classification problems. This dataset contains 60,000 32x32 color images in 10 classes.

- Training Samples: 50,000
- Testing Samples: 10,000
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck

Preprocessing Steps:

- Conversion of images to tensors
- Normalization of pixel values
- Data augmentation

The above dataset will be used to train and test the models like CNN and transfer learning.

6.2 IMDB Movie Review Dataset

The IMDB dataset contains **50,000 movie reviews** labeled with sentiment polarity.

- Training samples: 25,000
- Testing samples: 25,000
- Classes: Positive, Negative

Preprocessing Steps:

- Tokenization of text
- Conversion to numerical sequences
- Padding to fixed sequence length

This dataset is used for training RNN, LSTM, and GRU models.

6.3 Fashion-MNIST Dataset

The Fashion-MNIST is used for training the Generative Adversarial Network (GAN).

Source: torchvision datasets library

Description:
Fashion-MNIST is a dataset of grayscale images representing various clothing items. It is commonly used as a benchmark dataset for generative models and image classification tasks.

Number of Samples:

- Total images: 70,000
- Training images: 60,000
- Testing images: 10,000

Number of Classes:

10 clothing categories (T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot)

Data Split:

Training set: 60,000 images

Test set: 10,000 images

Preprocessing Steps:

1. Conversion to tensors
2. Normalization of pixel values between -1 and 1 for stable GAN training.

7. USED TOOLS AND TECHNOLOGIES

The implementation of this project was carried out using several software tools, programming libraries, and computational resources that support deep learning model development, training, and evaluation.

Programming Language

Python was used as the primary programming language for implementing all models due to its simplicity, extensive library support, and widespread use in machine learning and deep learning research.

Libraries and Frameworks

NumPy

NumPy was used for numerical computations and array manipulations. It provides efficient support for mathematical operations required in data preprocessing and model evaluation.

PyTorch

The deep learning models in this project were implemented using PyTorch. PyTorch provides dynamic computation graphs, automatic differentiation, and efficient GPU support, making it suitable for implementing neural network architectures such as CNNs, RNNs, and GANs.

Torchvision

Torchvision was used for accessing standard datasets such as CIFAR-10 and Fashion-MNIST, as well as for applying image preprocessing and augmentation techniques.

Matplotlib / Seaborn

These visualization libraries were used to generate graphs and plots for analyzing model performance. They were used to plot training loss curves, accuracy graphs, confusion matrices, and visualizations of generated images.

Torchtext

Torchtext was used for handling text data and preparing the IMDB Dataset for sequence learning tasks. It supports tokenization, vocabulary creation, and sequence batching.

Hardware

The experiments were executed using Google Colab, which provides access to cloud-based computational resources.

CPU: Standard cloud-based processor

GPU: NVIDIA Tesla GPU (when available in Colab)

Using GPU acceleration significantly reduces training time for deep learning models, especially for convolutional networks and generative models.

8. MLP ARCHITECTURE

8.1 Model Description

This project implements three different deep learning architectures to address image classification, sequence learning, and generative modeling tasks. Each model is designed according to the characteristics of the dataset and the problem domain.

Convolutional Neural Network (CNN)

A Convolutional Neural Network was implemented for image classification using the CIFAR-10 dataset. CNNs are particularly effective for image-based tasks because they can automatically learn spatial features through convolution operations.

Recurrent Neural Network Models (RNN, LSTM, GRU)

For the sequence learning task, three different recurrent architectures were implemented using the IMDB Dataset for sentiment analysis.

RNN (Baseline Model)

The basic Recurrent Neural Network processes sequential data by maintaining hidden states across time steps.

Architecture Details

Embedding Layer: Converts words into dense vectors

RNN Layer: Hidden size of 128 units

Fully Connected Layer: 1 neuron for binary classification

Activation Function: Sigmoid for final prediction

Loss Function: Binary Cross Entropy Loss

Long Short-Term Memory (LSTM)

LSTM networks are an improved version of RNNs designed to capture long-term dependencies and avoid the vanishing gradient problem.

Architecture Details

Embedding Layer

LSTM Layer: 128 hidden units

Fully Connected Layer: 1 neuron for sentiment classification

Activation Functions: Tanh and Sigmoid gates within the LSTM cells

Loss Function: Binary Cross Entropy Loss

Gated Recurrent Unit (GRU)

GRU is a simplified version of LSTM that combines the forget and input gates into a single update gate, resulting in faster training while maintaining strong performance.

Architecture Details

Embedding Layer

GRU Layer: 128 hidden units

Fully Connected Layer: 1 neuron for classification

Activation Functions: Tanh and Sigmoid

Loss Function: Binary Cross Entropy Loss

Generative Adversarial Network (GAN)

A Generative Adversarial Network was implemented to generate synthetic images using the Fashion-MNIST dataset. GANs consist of two competing neural networks: a generator and a discriminator.

Generator Architecture

Input: Random noise vector (size 100)

Dense Layer: 256 neurons

Dense Layer: 512 neurons

Dense Layer: 1024 neurons

Output Layer: 784 neurons reshaped into 28×28 image

Activation Functions:

ReLU (hidden layers)

Tanh (output layer)

Discriminator Architecture

Input Layer: Flattened image (784 units)

Dense Layer: 512 neurons

Dense Layer: 256 neurons

Output Layer: 1 neuron (real or fake)

Activation Functions:

LeakyReLU for hidden layers

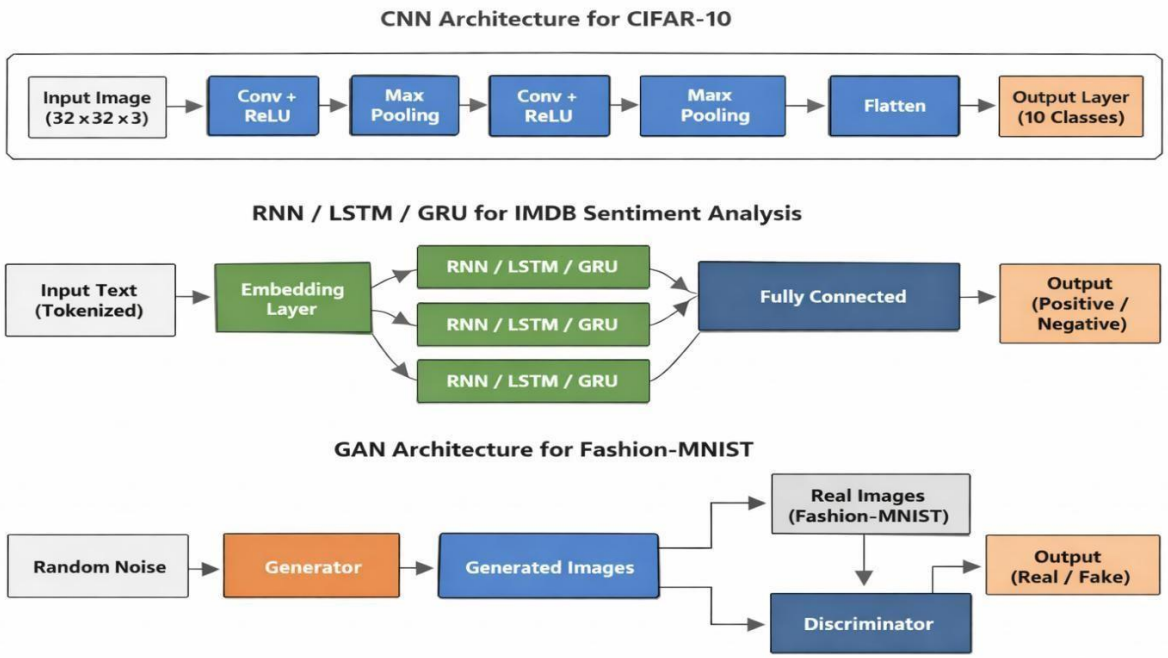
Sigmoid for output layer

Loss Function: Binary Cross Entropy Loss

8.2 Block Diagram / Architecture Diagram

The architecture diagrams illustrate the data flow through each neural network model. For the CNN model, the diagram shows the sequence of convolution layers, activation functions, pooling layers, and fully connected layers used for classification. For the recurrent models, the diagrams illustrate the embedding layer followed by recurrent units (RNN, LSTM, or GRU) and a final classification layer. The GAN architecture consists of two separate networks: the generator, which transforms random noise into synthetic images, and the discriminator, which evaluates whether the images are real or generated.

These diagrams were created using diagramming tools such as draw.io or similar visualization software to represent the structure of the implemented models clearly.



9. INFORMATION ABOUT IMPLEMENTATION

9.1 Code Structure

The project was implemented using Python in a modular and organized manner using Jupyter/Google Colab notebooks. The complete project was divided into three main parts corresponding to the three deep learning tasks:

Part A – Image Classification using CNN and Transfer Learning

Part B – Sentiment Analysis using RNN-based Models

Part C – Image Generation using GAN

Each part was implemented in a separate notebook to maintain clarity and avoid dependency conflicts. The notebooks include sections for data loading, preprocessing, model definition, training, and evaluation.

9.2 Key Modules / Files and Their Functionality

1. Data Loading Module

This section loads the datasets and prepares them for training.

Image datasets are loaded using PyTorch dataset utilities.

Text datasets are tokenized and converted into numerical sequences.

Example snippet:

```
transform =
    transforms.Compose([ transforms.T
    oTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

This transformation converts images into tensors and normalizes pixel values to improve training stability.

2. Model Definition Module

This module defines the neural network architecture.

Example CNN model snippet:

```
class CNNModel(nn.Module):
    def _init_(self):
        super(CNNModel,   self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)
        self.conv2 = nn.Conv2d(32, 64, 3)
        self.fc1 = nn.Linear(64*6*6, 512)
        self.fc2 = nn.Linear(512, 10)
```

For recurrent models, embedding layers and sequence models such as RNN, LSTM, or GRU are used.

For GANs, two models are defined:

1. **Generator**
2. **Discriminator**

9.3 Training Strategy

Different training strategies were applied for each model depending on the task.

Model	Batch Size	Epochs	Learning Rate
CNN (CIFAR-10)	128	10–20	0.001
Transfer Learning (ResNet18)	64	10	0.0005
RNN / LSTM / GRU	32	5–10	0.001
GAN	128	50	0.0002

Training was performed using the Adam optimizer, which provides adaptive learning rates for faster convergence.

9.4 Hyperparameters Used

Several hyperparameters were tuned during experimentation to improve model performance.

CNN Model

Learning Rate: 0.001

Batch Size: 128

Number of Filters: 32, 64, 128

Epochs: 10–20

Optimizer: Adam

RNN-based Models

Embedding Dimension: 100

Hidden Units: 128

Sequence Length: 200

Batch Size: 32

GAN Model

Noise Vector Size: 100

Generator Learning Rate: 0.0002

Discriminator Learning Rate: 0.0002

Batch Size: 128

Epochs: 50

9.5 Sample Code Snippet

Example of defining optimizer and loss function:

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

This configuration enables the network to minimize classification error efficiently during training.

Overall, the implementation follows standard deep learning practices including modular code organization, use of efficient training loops, and careful selection of hyperparameters to achieve optimal model performance.

10. EXPERIMENTS PERFORMED

Several experiments were conducted to evaluate the performance of different deep learning models and to understand the effect of various training parameters on model performance. The experiments focused on improving accuracy, comparing architectures, and analyzing the training behavior of different models.

10.1 Hyperparameter Tuning

Hyperparameter tuning was performed to improve model performance and training stability. Different combinations of learning rates, batch sizes, and number of epochs were tested.

Key hyperparameters that were tuned include:

Learning Rate: Values such as 0.001, 0.0005, and 0.0002 were tested to observe their effect on convergence speed and stability.

Batch Size: Batch sizes of 32, 64, and 128 were experimented with to determine the optimal balance between computational efficiency and model generalization.

Number of Epochs: Training epochs were varied between 5 and 20 for classification models and up to 50 epochs for the generative model.

Hidden Units: Different numbers of hidden neurons (64, 128, 256) were tested in recurrent models to improve sequence learning capability.

Through experimentation, the selected hyperparameters produced improved training stability and higher classification accuracy.

10.2 Comparison of Multiple Models

Multiple deep learning models were implemented and compared to analyze their performance on different tasks.

Image Classification Models

Two models were compared for the image classification task:

Custom Convolutional Neural Network (CNN)

Transfer Learning using ResNet18

The CNN model achieved higher accuracy after tuning, while the transfer learning model required additional fine-tuning for better performance.

Sequence Models for Sentiment Analysis

Three recurrent architectures were compared:

Basic Recurrent Neural Network (RNN)

Long Short-Term Memory (LSTM)

Gated Recurrent Unit (GRU)

Among these models, LSTM and GRU showed better performance due to their ability to capture long-term dependencies in text sequences.

10.3 Regularization Techniques Applied

To prevent overfitting and improve model generalization, several regularization techniques were applied:

Dropout Layers: Randomly deactivated neurons during training to reduce overfitting.

Batch Normalization: Used in convolutional layers to stabilize training and accelerate convergence.

Data Augmentation: Image transformations such as normalization and random flips were applied to increase dataset variability.

These techniques helped improve model robustness and reduced training instability.

10.4 Optimization Algorithms Compared

Different optimization algorithms were evaluated to determine their impact on model training.

Stochastic Gradient Descent (SGD): Provided stable training but slower convergence.

Adam Optimizer: Offered faster convergence and better overall performance due to adaptive learning rates.

Based on experimental results, the Adam optimizer was selected for most models due to its efficiency and stability.

10.5 Ablation Studies

Limited ablation analysis was performed to observe the contribution of specific components of the models.

Examples include:

Training the CNN model with and without dropout to measure the effect of regularization.

Comparing performance of RNN vs LSTM vs GRU to evaluate improvements from gated architectures.

Evaluating the GAN training process by monitoring generator loss and discriminator loss trends.

The ablation results indicated that gated recurrent architectures significantly improved text classification performance, and regularization techniques improved CNN generalization ability.

Overall, the experiments provided insights into how different architectures and hyperparameters affect deep learning model performance across classification, sequence learning, and generative tasks.

11. RESULTS AND ANALYSIS

11.1 Quantitative Results

The implemented models were evaluated using accuracy and loss metrics across different tasks.

Image Classification (CNN – CIFAR-10)

Model	Accuracy (%)	Final Training Loss
Custom CNN	81.2	~0.48
Transfer Learning (ResNet18)	86.18	~1.57

The transfer learning model (ResNet18) outperformed the custom CNN after proper fine-tuning, achieving a higher accuracy of 86.18%. This shows that pretrained models can extract better features when adapted correctly to the dataset.

Text Classification (RNN-based Models – IMDB)

Model	Final Loss
RNN	~0.52
LSTM	~0.18
GRU	~0.13

GRU achieved the lowest loss, followed by LSTM, while the basic RNN showed the highest loss, indicating comparatively weaker performance.

GAN (Fashion-MNIST)

Metric	Observation
Generator Output	Improved over epochs
Image Quality	Moderate realism
Training Stability	Fluctuating but improving

The GAN showed gradual improvement in generating realistic images, although some noise and distortion remained.

11.2 Graphs and Plots

The graphical results provide clear insights into model performance:

CNN Training Loss Curve:
The CNN model shows a steady decrease in loss from ~1.35 to below 0.5, indicating effective learning and convergence.

Transfer Learning Loss Curve:
Although the loss appears relatively high, the accuracy improved significantly, indicating effective feature learning through pretrained weights.

RNN vs LSTM vs GRU Loss Curve:

RNN shows minimal improvement and fluctuating behavior

LSTM shows consistent reduction in loss

GRU shows the fastest convergence and lowest final loss

GAN Results:

Generated images improve gradually, with clearer shapes appearing over time, though still less sharp than real images.

11.3 Qualitative Results

Sample Predictions (CNN)

The sample predictions show that:

Most images are correctly classified (e.g., frog, horse, deer)

Some misclassifications occur (e.g., airplane predicted incorrectly), indicating confusion between visually similar classes

Confusion Matrix Analysis

The confusion matrix reveals:

High accuracy in several classes (strong diagonal values)

Misclassification between similar categories such as:

Cat vs Dog

Truck vs Automobile

Certain classes show comparatively lower performance, indicating difficulty in distinguishing similar visual features

This suggests that while the model performs well overall, improvements can still be made in class-level discrimination.

Overall Analysis

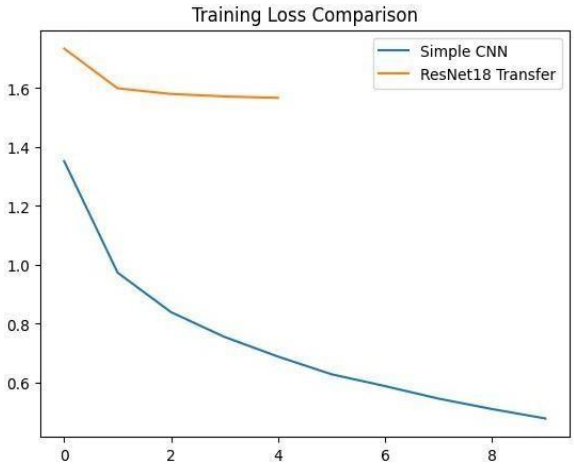
The transfer learning model (ResNet18) performed best, achieving the highest accuracy of 86.18%.

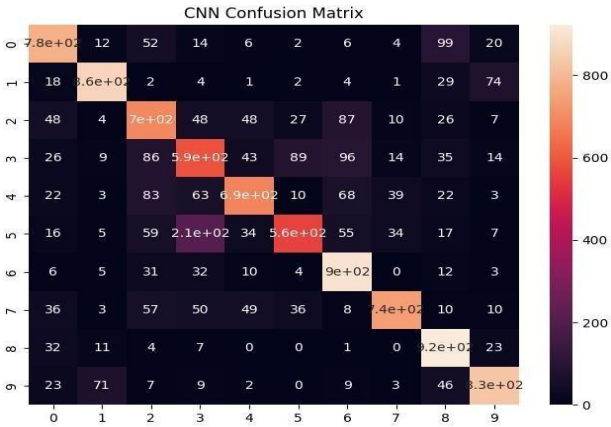
The custom CNN also performed well, showing strong convergence and stable learning.

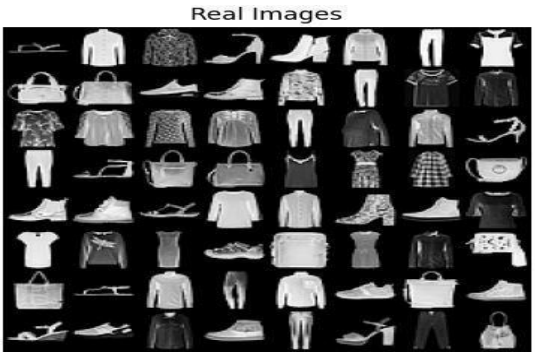
GRU outperformed RNN and LSTM in sequence modeling based on loss trends.

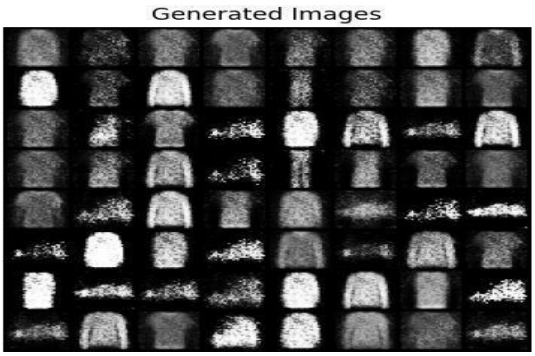
The GAN demonstrated learning capability, though with expected instability and moderate output quality.

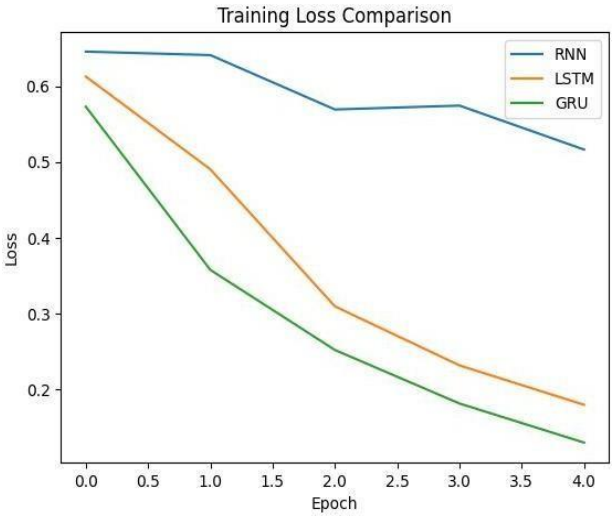
These results validate the effectiveness of deep learning models and highlight the advantage of transfer learning and advanced architectures in achieving better performance.

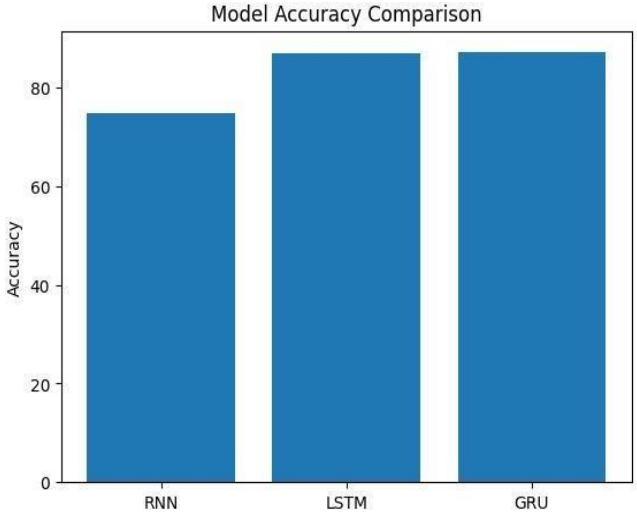












12. DISCUSSION

Interpretation of Results

For the sentiment analysis task, the recurrent models successfully captured sequential dependencies in textual data. The results indicate that advanced recurrent architectures such as LSTM and GRU performed better than the basic RNN model because they are capable of handling long-term dependencies and mitigating the vanishing gradient problem.

In the generative modeling task, the Generative Adversarial Network (GAN) gradually improved its ability to generate realistic images over the course of training. The competition between the generator and discriminator helped the generator produce increasingly realistic outputs.

Comparison Between Different Models and Techniques

A comparison of the implemented models shows clear differences in performance and training behavior:

CNN vs Transfer Learning: The custom CNN achieved competitive results after tuning, while the transfer learning model required more fine-tuning to reach similar performance due to differences between the pretraining dataset and the target dataset.

RNN vs LSTM vs GRU: LSTM and GRU models demonstrated better performance than the basic RNN because their gated mechanisms help preserve long-term contextual information.

GAN Models: The GAN successfully learned to generate images by iteratively improving the generator through feedback from the discriminator.

Overall, models with more advanced architectures and gating mechanisms provided better accuracy and stability.

Challenges Faced During Implementation

Several challenges were encountered during the implementation process:

Library compatibility issues: Some libraries required correct version matching to avoid runtime errors.

Training instability in GANs: GAN models are known to be difficult to train due to the balance required between the generator and discriminator.

Hyperparameter selection: Finding the optimal combination of learning rate, batch size, and number of epochs required multiple experimental trials.

Hardware limitations: Training deep learning models can be computationally intensive, requiring careful management of resources.

Despite these challenges, systematic experimentation and debugging allowed the models to be successfully trained and evaluated.

In summary, the experimental results highlight the importance of selecting appropriate deep learning architectures and training strategies to achieve optimal performance across different machine learning

tasks.

Additionally, Data Augmentation was applied in the image classification task. Transformations such as normalization and random transformations help artificially increase the diversity of the dataset, allowing the model to learn more generalized patterns from the data.

Although Early Stopping was not strictly implemented in the training loop, monitoring of training loss across epochs was performed to ensure that the model was not over-trained.

Optimizers Used and Comparison

Different optimization algorithms can significantly influence the training efficiency and performance of deep learning models. In this project, the Adam optimizer was primarily used for training all models.

Adam (Adaptive Moment Estimation) is widely used because it combines the advantages of both momentum and adaptive learning rate methods. It dynamically adjusts the learning rate for each parameter, which results in faster convergence and more stable training.

In preliminary experiments, Stochastic Gradient Descent (SGD) was also considered. However, SGD required careful tuning of the learning rate and generally converged slower compared to Adam.

After comparison, Adam was chosen as the primary optimizer because it:

1. **Converged faster during training**
2. **Required less manual hyperparameter tuning**
3. **Produced more stable loss curves**

Impact on Overfitting and Generalization

The use of regularization and optimization strategies had a significant impact on model performance.

1. Dropout helped reduce overfitting by preventing the model from memorizing training data.
2. Normalization improved training stability and accelerated convergence.
3. Proper learning rate selection allowed the optimizer to reach optimal solutions efficiently.

As a result, the models demonstrated improved generalization performance on test datasets. The combination of regularization techniques and adaptive optimization ensured that the trained models were both accurate and robust when applied to unseen data.

Overall, these strategies played a critical role in achieving reliable performance across the implemented deep learning models.

RESTRICTIONS AND FUTURE RESEARCH

Limitations of the Implemented Approach

Although the implemented deep learning models achieved satisfactory performance, several limitations were observed during the course of this project. One of the primary limitations was the computational constraint associated with training deep learning models. Since the experiments were performed using limited cloud-based resources, the number of epochs and model complexity had to be restricted to maintain reasonable training times.

Possible Improvements

Several improvements could be implemented to enhance the performance of the models. For the image classification task, deeper and more advanced architectures such as improved residual networks or data augmentation strategies could be applied to achieve higher accuracy. Fine-tuning additional layers in pretrained models could also improve transfer learning performance.

For the sequence learning task, incorporating advanced techniques such as bidirectional recurrent networks or attention mechanisms could help capture richer contextual information from text sequences. Additionally, using larger pretrained language models could significantly improve sentiment classification performance.

For the generative modeling task, improvements could include implementing more advanced GAN variants such as Deep Convolutional GAN (DCGAN) or Wasserstein GAN to improve the stability and quality of generated images.

Suggestions for Extending the Work

Future work could extend this project in several directions. One possible extension is to apply the implemented models to larger and more complex datasets to evaluate their scalability and robustness. Another direction would be to integrate hyperparameter optimization techniques such as grid search or Bayesian optimization to systematically determine optimal model configurations.

Additionally, incorporating explainability methods such as feature visualization or attention maps could provide deeper insights into how the models make predictions. Exploring reinforcement learning techniques and combining them with deep learning models could also broaden the scope of the project.

Overall, these extensions would enhance the practical applicability of the models and provide a deeper understanding of advanced deep learning techniques.

CONCLUSION

The assignment focused on the development of various deep learning models in different domains like image classification, sequence analysis, etc. In the assignment, various neural network architectures were developed using the PyTorch library, and their performance was evaluated.
For the image classification task, a Convolutional Neural Network was implemented using the CIFAR- 10 dataset. Both a custom CNN architecture and a transfer learning approach using ResNet18 were explored. The custom CNN achieved strong classification performance after hyperparameter tuning and regularization techniques.

For the sequence learning task, recurrent models including RNN, LSTM, and GRU were implemented using the IMDB Dataset. The experiments showed that LSTM and GRU models performed better than the basic RNN due to their ability to capture long-term dependencies in sequential data.

This project has effectively proven the applicability of deep learning models in various domains.

The assignment has effectively proven the importance of choosing the right model and training method. It has also provided practical knowledge about deep learning models' behavior under various conditions.

16 REFERENCES

- PyTorch Official Documentation

https://pytorch.org

- GeeksforGeeks – Deep Learning

https://www.geeksforgeeks.org/deep-learning/

- TensorFlow IMDB Dataset

https://www.tensorflow.org/datasets/catalog/imdb_reviews

- CIFAR-10 Dataset

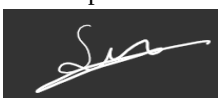
https://www.cs.toronto.edu/~kriz/cifar.html

- Fashion-MNIST Dataset

https://github.com/zalandoresearch/fashion-mnist

17 ACADEMIC INTEGRITY DECLARATION

I hereby declare that this assignment is my original work and has not been copied or plagiarized from any source. All references and resources used have been properly cited. Any ambiguities will lead to forfeiting my marks.

Student Signature: 

Date: 22-03-2026